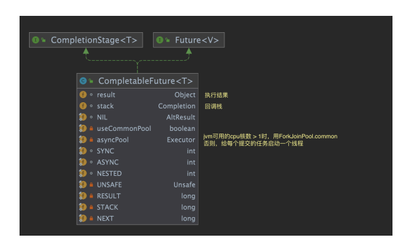

```java
public class CompletableFutureTest { //CompletableFuture使用示例，假设当前线程为「主线程」
  public static void main(String[] args) throws Exception {
    //2.调用CompletableFuture静态方法，异步执行
    CompletableFuture<String> supplyCF = CompletableFuture.supplyAsync(CompletableFutureTest::evenNumbersSum);

    //3.设置回调方法
    supplyCF.thenApply(r - > { return r+1; })
            .thenApply(r - > { return r+1; })
            .thenAccpet(System.out::println);

    //4.设置异常时的回调方法（如果没有异常，返回的d.result和supplyCF.result相同）
    supplyCF.exceptionally();

    //主线程先跑着
    for (int i = 1; i <= 10; i++) {
      System.out.println("main Thread print: " + i);
      Thread.sleep(50);
    }
  }
  private static Integer evenNumbersSum() { //1.定义业务逻辑
    int sum = 0;
    for (int i = 1; i <= 100; i++) {
      if(i%2 == 0) sum += i; //偶数相加
    }
    return sum; //返回结果
  }
}
```

```java
supplyAsync(Supplier<U> supplier)
→ asyncSupplyStage(Executor e, Supplier<U> f) {
  if (f == null) throw new NullPointerException();
  CompletableFuture<U> d = new CompletableFuture<U>();
  e.execute(new AsyncSupply<U>(d, f)); //任务提交线程池
  return d; //3.直接返回
}
```

```java
/* 线程池中某个「工作线程」在workQueue中取出task，并执行
ForkJoinWorkerThread.run() → runWorker() → runTask() → doExec() → AsyncSupply.exec() → AsyncSupply.run() {
  CompletableFuture<T> d; Supplier<T> f;
  if ((d = dep) != null && (f = fn) != null) {
    dep = null; fn = null;
    if (d.result == null) {
      try {
        d.completeValue(f.get()); //1.执行CompletableFutureTest.evenNumbersSum()，并将结果保存到d.result
      } catch (Throwable ex) { d.completeThrowable(ex); }
    }
    d.postComplete(); //2.异步执行回调函数
  }
}
```

```java
boolean completeValue(T t) { //将结果存入d.result
  return UNSAFE.compareAndSwapObject(this, RESULT, null, (t == null) ? NIL : t);
}
boolean completeThrowable(Throwable x) { //将异常存入d.result
  return UNSAFE.compareAndSwapObject(this, RESULT, null, encodeThrowable(x));
}
```

```java
void postComplete() { //执行d.stack中所有待执行的回调函数
  CompletableFuture<?> f = this; Completion h;
  while ((h = f.stack) != null ||
         (f != this && (h = (f = this).stack) != null)) { //stack第1个节点x不为空
    CompletableFuture<?> d; Completion t;
    if (f.casStack(h, t = h.next)) { //x移出stack
      if (t != null) { //stack新的第1个节点y不为空
        if (f != this) {
          pushStack(h); //x
          continue;
        }
        h.next = null;
      }
      f = (d = h.tryFire(NESTED)) == null ? this : d; //执行回调逻辑
    }
  }
}
```

```java
<U> CompletableFuture<U> thenApply(Function<? super T,? extends U> fn) {
  return uniApplyStage(null, fn);
}

<V> CompletableFuture<V> uniApplyStage(Executor e, Function<? super T,? extends U> f) {
  if (f == null) throw new NullPointerException();
  CompletableFuture<Void> d = new CompletableFuture<Void>();
  if (e != null || !d.uniApply(this, f, null)) { //1.直接执行回调逻辑，如果成功，直接返回
    //2.this.result还没出来 or caller指定了executor：将回调逻辑封装一下，压入this.stack
    UniApply<T,V> c = new UniApply<T,V>(e, d, this, f);
    push(c);
    c.tryFire(SYNC); //3.再试一次同步执行
  }
  return d;
}
```

```java
boolean uniApply(CompletableFuture<S> a, Function<? super S,? extends T> f, UniApply<S,T> c) {
  Object r; Throwable x;
  if (a == null || (r = a.result) == null || f == null) return false; //上一步还没执行完
  tryComplete:
  if (result == null) {
    if (r instanceof AltResult) { //上一步异常
      if ((x = ((AltResult)r).ex) != null) {
        completeThrowable(x, r);
        break tryComplete; //结束
      }
      r = null;
    }
    try {
      if (c != null && !c.claim()) return false;
      S s = (S) r;
      completeValue(f.apply(s)); //执行回调action，入参是上一步action的执行结果
    } catch (Throwable ex) { completeThrowable(ex); }
  }
  return true;
}
```

```java
CompletableFuture<Void> tryFire(int mode) {
  CompletableFuture<Void> d; CompletableFuture<T> a;
  if ((d = dep) == null ||
      !d.uniApply(a = src, fn, mode > 0 ? null : this)) return null;
  dep = null; src = null; fn =  null;
  return d.postFire(a, mode);
}
```

```java
void push(UniCompletion<?,?> c) {
  if (c != null) {
    while (result == null && !tryPushStack(c)) //1.尝试入栈，直到result不为空
      lazySetNext(c, null); //2.入栈失败，清空c.next
  }
}
```

```java
CompletableFuture<T> postFire(CompletableFuture<?> a, int mode) {
  if (a != null && a.stack != null) {
    if (mode < 0 || a.result == null) a.cleanStack();
    else a.postComplete();
  }
  if (result != null && stack != null) {
    if (mode < 0) return this;
    else postComplete();
  }
  return null;
}
```

```java
boolean tryPushStack(UniCompletion c) {
  Completion h = stack;
  lazySetNext(c, h); //c.next指向stack第1个元素
  return UNSAFE.compareAndSwapObject(this, STACK, h, c); //入栈
}
```

```java
CompletableFuture<Void> thenAccept(Consumer<? super T> action) {
  return uniAcceptStage(null, action);
}

CompletableFuture<Void> uniAcceptStage(Executor e, Consumer<? super T> f) {
  if (f == null) throw new NullPointerException();
  CompletableFuture<Void> d = new CompletableFuture<Void>();
  if (e != null || !d.uniAccept(this, f, null)) { //1.直接执行回调逻辑，如果成功，直接返回
    //2.this.result还没出来 or caller指定了executor：将回调逻辑封装一下，压入this.stack
    UniAccept<T> c = new UniAccept<T>(e, d, this, f);
    push(c);
    c.tryFire(SYNC);
  }
  return d;
}
```

```java
boolean uniAccept(CompletableFuture<S> a, Consumer<? super S> f, UniAccept<S> c) {
  Object r; Throwable x;
  if (a == null || (r = a.result) == null || f == null) return false; //获取上一步action的执行结果
  tryComplete:
  if (result == null) {
    if (r instanceof AltResult) { //上一步异常
      if ((x = ((AltResult)r).ex) != null) {
        completeThrowable(x, r);
        break tryComplete; //结束
      }
      r = null;
    }
    try {
      if (c != null && !c.claim()) return false;
      S s = (S) r;
      f.accept(s); //执行回调action，入参是上一步action的执行结果
      completeNull(); //设置结果
    } catch (Throwable ex) { completeThrowable(ex); }
  }
  return true;
}
```

```java
CompletableFuture<T> exceptionally(Function<Throwable, ? extends T> fn) {
  return uniExceptionallyStage(fn);
}

CompletableFuture<T> uniExceptionallyStage(Function<Throwable, ? extends T> f) {
  if (f == null) throw new NullPointerException();
  CompletableFuture<Void> d = new CompletableFuture<Void>();
  if (!d.uniExceptionally(this, f, null)) { //1.直接执行回调逻辑，如果成功，直接返回
    //2.this.result还没出来 or caller指定了executor：将回调逻辑封装一下，压入this.stack
    UniExceptionally<T> c = new UniExceptionally<T>(d, this, f);
    push(c);
    c.tryFire(SYNC); //3.再试一次同步执行
  }
  return d;
}
```

```java
boolean uniExceptionally(CompletableFuture<S> a,
                         Function<? super Throwable, ? extends T> f, UniExceptionally<T> c) {
  Object r; Throwable x;
  if (a == null || (r = a.result) == null || f == null) return false; //获取上一步action的执行结果
  tryComplete:
  if (result == null) {
    try {
      if (r instanceof AltResult && (x = ((AltResult)r).ex) != null) { //确认执行过程发生异常
        if (c != null && !c.claim()) return false;
        completeValue(f.apply(x)); //执行回调action，入参是异常x
      } else internalComplete(r); //没有异常，CAS from null to r (completes normally with the same value)
    } catch (Throwable ex) { completeThrowable(ex); }
  }
  return true;
}
```